

Persistencia

Laboratorio 6

Oscar Lasso

Juan Diego Gaitán

Profesor:

Maria Irma Diaz Rozo

Juan Frasca

Desarrollo Orientado a Programación de Objetos

Escuela Colombiana de Ingeniería Julio Garavito

Bogotá D.C.

OBJETIVO

Evidenciar el logro de los siguientes resultados de aprendizaje:

1. Extender el código de un proyecto considerando nuevos requisitos funcionales.
2. Diseñar y construir los métodos básicos de manejo de archivos: guardar como, abrir, exportar como e importar.
3. Controlar las excepciones generadas al trabajar con archivos.
4. Experimentar las prácticas

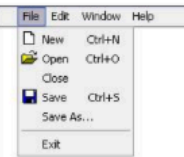
Coding Only one pair [integrates code at a time](#).

Coding. Use [collective ownership](#).

CONTEXTO

Reto: Extender el proyecto [forest](#) adicionando el menú Abrir en el menú barra con las opciones básicas de entrada-salida (Abrir - Guardar como e Importar - Exportar como) y las opciones estándar Nuevo y Salir.

El menú Archivo reúne las funciones básicas de gestión de documentos organizadas de forma consistente y predecible.
Las guías de *Apple* y las directrices de *Microsoft* para el diseño de interfaz establecen que esta estructura y semántica deben mantenerse estables para garantizar la usabilidad, constituyendo así un estándar de facto en las interfaces gráficas modernas.



PARTE I. Creando la maqueta [En [lab06.doc](#) [forest.astah](#) *.java]

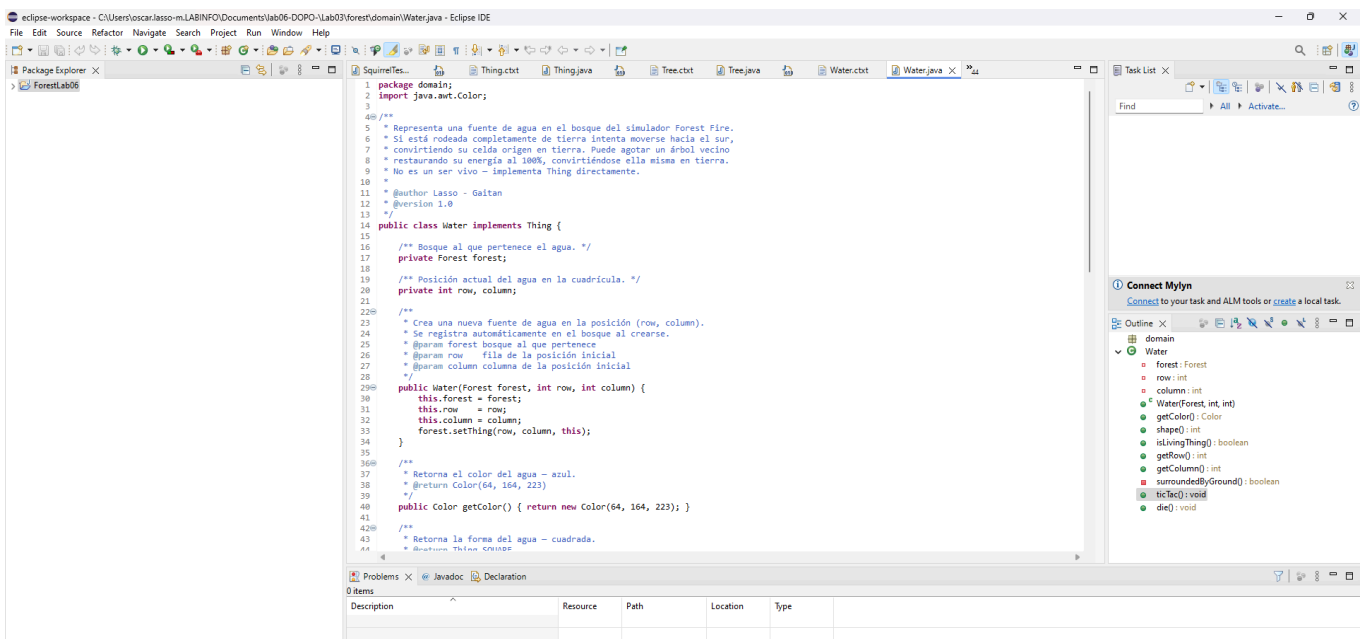
(NO OLVIDEN BDD – MDD)

En este punto vamos a construir la maqueta correspondiente a esta extensión siguiendo el patrón MVC.

A [forest](#) inicial

1. En su directorio descarguen el laboratorio 03 realizado por ustedes y seleccionen el ambiente para trabajar: [BlueJ](#) o [Eclipse](#). ¿Cuál ambiente seleccionaron? ¿Por qué?
2. Ejecuten el programa y capturen la pantalla.

1. Decidimos seleccionar el ambiente de trabajo Eclipse debido a su interfaz y facilidad para realizar test. Consideramos que este IDE es completo y más apropiado que [BLUEJ](#) para un proyecto profesional.



B Maqueta: MVC

1. **MODELO:** Preparen en la clase fachada del dominio los métodos correspondientes a las cuatro opciones básicas de entrada-salida (`open`, `saveAs`, `import` y `exportAs`). Estos métodos deben tener un parámetro `File` y simplemente deben propagar una `ForestException` con el mensaje de "Opción nombreOpción en construcción. Archivo nombreArchivo".
Para la opción Nuevo se usará el constructor de la clase `Forest` y la opción Salir no requiere del modelo.

```
1 package domain;
2
3 /**
4  * Excepción personalizada para los errores del dominio Forest.
5  *
6  * @author Oscar Lasso - Juan Diego Gaitan
7  * @version 1.0
8  */
9 public class ForestException extends Exception {
10
11     public static final String OPCION_OPEN_EN_CONSTRUCCION = "Opción open en construcción. Archivo ";
12     public static final String OPCION_SAVE_AS_EN_CONSTRUCCION = "Opción saveAs en construcción. Archivo ";
13     public static final String OPCION_IMPORT_EN_CONSTRUCCION = "Opción import en construcción. Archivo ";
14     public static final String OPCION_EXPORT_AS_EN_CONSTRUCCION = "Opción exportAs en construcción. Archivo ";
15
16     public ForestException(String message) {
17         super(message);
18     }
19 }
20
```

```
96     public void open(File file) throws ForestException {
97         throw new ForestException(ForestException.OPCION_OPEN_EN_CONSTRUCCION + file.getName());
98     }
99
100     public void saveAs(File file) throws ForestException {
101         throw new ForestException(ForestException.OPCION_SAVE_AS_EN_CONSTRUCCION + file.getName());
102     }
103
104     public void importFile(File file) throws ForestException {
105         throw new ForestException(ForestException.OPCION_IMPORT_EN_CONSTRUCCION + file.getName());
106     }
107
108     public void exportAs(File file) throws ForestException {
109         throw new ForestException(ForestException.OPCION_EXPORT_AS_EN_CONSTRUCCION + file.getName());
110     }
111 }
```

2. **VISTA** : Construyan el menú Archivo que ofrezca, además de las opciones básicas de entrada-salida, las opciones estándar de nuevo y salir (Nuevo, Abrir, Salvar como, Importar, Exportar como y Salir). No olviden incluir los separadores.
Para esto creen el método `prepareElementsMenu`. Capturen la pantalla del menú.

```
/**
 * Construye y configura el menú superior de la ventana.
 */
private void prepareElementsMenu() {
    JMenuBar menuBar = new JMenuBar();
    setJMenuBar(menuBar);

    JMenu menuArchivo = new JMenu("Archivo");
    menuBar.add(menuArchivo);

    itemNuevo = new JMenuItem("Nuevo");
    itemAbrir = new JMenuItem("Abrir");
    itemSalvarComo = new JMenuItem("Salvar como");
    itemImportar = new JMenuItem("Importar");
    itemExportarComo = new JMenuItem("Exportar como");
    itemSalir = new JMenuItem("Salir");

    menuArchivo.add(itemNuevo);
    menuArchivo.add(itemAbrir);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalvarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemImportar);
    menuArchivo.add(itemExportarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);
}
```

3. **CONTROLADOR:** Construyan los oyentes correspondientes a las seis opciones. Para esto creen el método `prepareActionsMenu` y los métodos base del controlador (`optionOpen`, `optionSaveAs`, `optionImport`, `optionExportAs`, `optionNew`, `optionExit`).
- a) Salir: El método `optionExit` que hace que se termine la aplicación. No es necesario incluir confirmación.
 - b) Nuevo: El método `optionNew` que crea un nuevo `forest`. Capturen una pantalla significativa.
 - c) Entrada-salida: Los métodos usan un `FileChooser` y atienden la excepción. Estos métodos llaman el método correspondiente del modelo que, por ahora, sólo lanzan una excepción. Ejecuten las diferentes acciones del menú y para cada una de ellas capture una pantalla significativa.

```
/**
 * Construye y configura el menú superior de la ventana.
 */
private void prepareElementsMenu() {
    JMenuBar menuBar = new JMenuBar();
    setJMenuBar(menuBar);

    JMenu menuArchivo = new JMenu("Archivo");
    menuBar.add(menuArchivo);

    itemNuevo = new JMenuItem("Nuevo");
    itemAbrir = new JMenuItem("Abrir");
    itemSalvarComo = new JMenuItem("Salvar como");
    itemImportar = new JMenuItem("Importar");
    itemExportarComo = new JMenuItem("Exportar como");
    itemSalir = new JMenuItem("Salir");

    menuArchivo.add(itemNuevo);
    menuArchivo.add(itemAbrir);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalvarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemImportar);
    menuArchivo.add(itemExportarComo);
    menuArchivo.addSeparator();
    menuArchivo.add(itemSalir);
}
```

```

/**
 * Enlaza los botones del menú con sus respectivas acciones.
 */
private void prepareActionsMenu() {
    itemNuevo.addActionListener(e -> optionNew());
    itemAbrir.addActionListener(e -> optionOpen());
    itemSalvarComo.addActionListener(e -> optionSaveAs());
    itemImportar.addActionListener(e -> optionImport());
    itemExportarComo.addActionListener(e -> optionExportAs());
    itemSalir.addActionListener(e -> optionExit());
}

/**
 * Crea un bosque nuevo y actualiza la interfaz.
 */
private void optionNew() {
    theForest = new Forest();
    photo.repaint();
}

/**
 * Permite seleccionar un archivo para abrir un bosque existente.
 */
private void optionOpen() {
    JFileChooser fileChooser = new JFileChooser();
    if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        try {
            theForest.open(fileChooser.getSelectedFile());
        } catch (ForestException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Ocurrió un error inesperado.", "Fatal Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

```

/**
 * Permite seleccionar un archivo para importar una configuración.
 */
private void optionImport() {
    JFileChooser fileChooser = new JFileChooser();
    if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        try {
            theForest.importFile(fileChooser.getSelectedFile());
        } catch (ForestException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Ocurrió un error inesperado.", "Fatal Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

/**
 * Permite seleccionar dónde exportar el estado del bosque.
 */
private void optionExportAs() {
    JFileChooser fileChooser = new JFileChooser();
    if (fileChooser.showSaveDialog(this) == JFileChooser.APPROVE_OPTION) {
        try {
            theForest.exportAs(fileChooser.getSelectedFile());
        } catch (ForestException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Ocurrió un error inesperado.", "Fatal Error", JOptionPane.ERROR_MESSAGE);
        }
    }
}

```

PARTE II. Implementando persistencia [En lab06.doc forest.astah *.java]

(NO OLVIDEN BDD – MDD)

A Guardar y Abrir

Las opciones guardar y abrir van a ofrecer servicios de persistencia del **forest** como objeto. Los nombres de los archivos deben tener como extensión **.dat**.

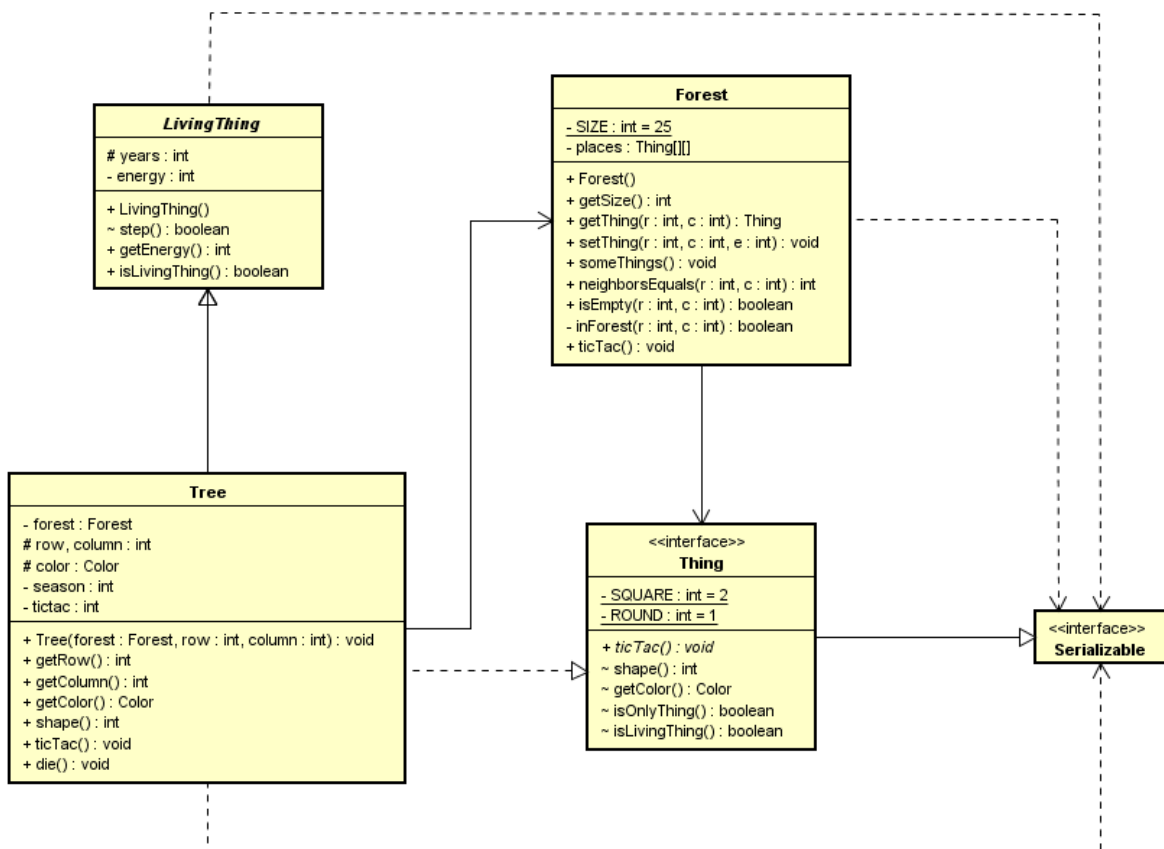
1. Copien las versiones actuales de **open** y **saveAs** y renómbrenlos como **open00** y **saveAs00**

```
/**
 * Abre un bosque desde un archivo.
 */
public void open00(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_OPEN_EN_CONSTRUCCION + file.getName());
}

/**
 * Abre un bosque desde un archivo.
 */
public void open(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_OPEN_EN_CONSTRUCCION + file.getName());
}

/**
 * Guarda el bosque actual en el archivo especificado (versión inicial).
 */
public void saveAs00(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_SAVE_AS_EN_CONSTRUCCION + file.getName());
}

/**
 * Guarda el bosque actual en el archivo especificado.
 */
public void saveAs(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_SAVE_AS_EN_CONSTRUCCION + file.getName());
}
```



```
/**
 * @author Lasso - Gaitan
 * @version 1.0
 */
public interface Thing extends Serializable {

    public static final int SQUARE = 2;

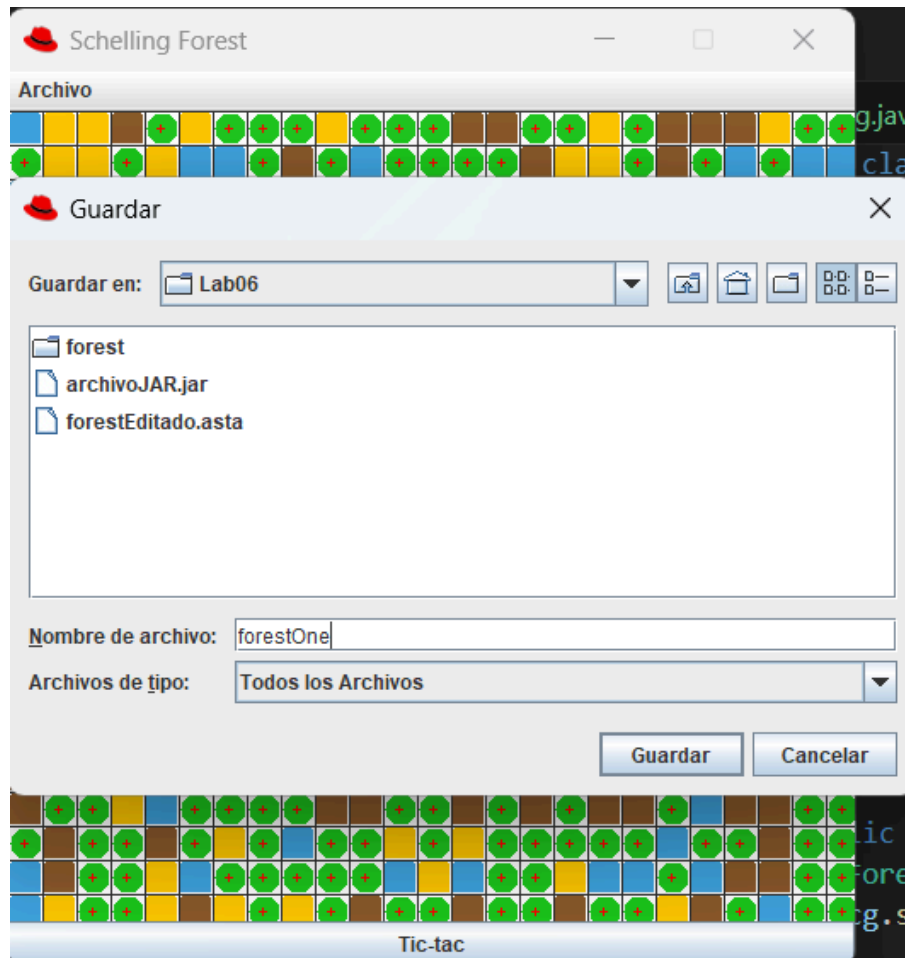
    public static final int ROUND = 1;
}
```

2. Construyan el método `saveAs` que ofrece el servicio de guardar en un archivo el estado actual de `forest`. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.

```
/**
 * Guarda el bosque actual en el archivo especificado usando serialización.
 */
public void saveAs(File file) throws ForestException {
    try (java.io.ObjectOutputStream out = new java.io.ObjectOutputStream(new java.io.FileOutputStream(file))) {
        out.writeObject(this);
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_GUARDAR);
    }
}
```

```
@Test
public void testSaveAs() {
    try {
        Forest testForest = new Forest(true);
        java.io.File tempFile = java.io.File.createTempFile("testForestSave", ".dat");
        testForest.saveAs(tempFile);
        assertTrue("El archivo debería haber sido creado", tempFile.exists());
        assertTrue("El archivo no debería estar vacío", tempFile.length() > 0);
        tempFile.delete(); // Limpiar el archivo después de la prueba
    } catch (Exception e) {
        fail("No debería haber lanzado excepción: " + e.getMessage());
    }
}
```


3. Validen este método guardando el estado obtenido después de dos clics como **forestOne.dat**.
¿El archivo se creó en el disco? ¿Cuánto espacio ocupa?

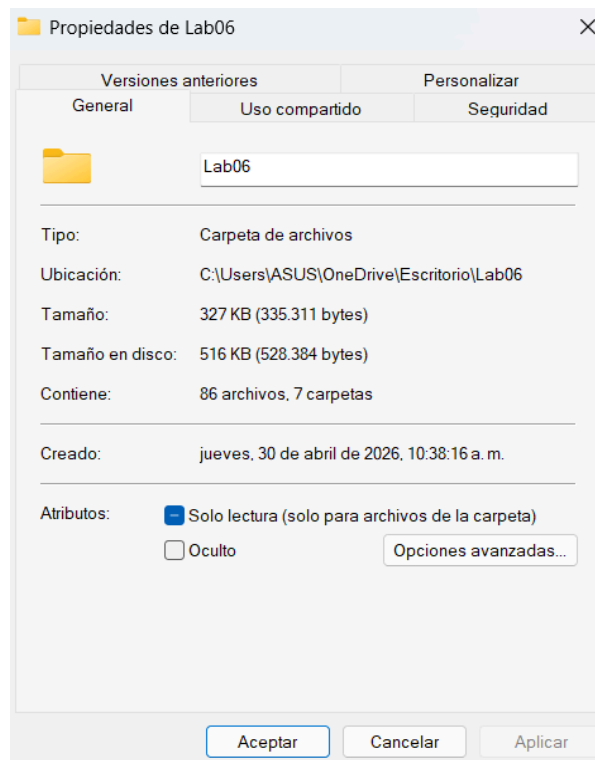


¿El archivo se creó en el disco?

- Sí

¿Cuánto espacio ocupa?

- 327 KB



4. Construyan el método `open` que ofrece el servicio de leer de un archivo. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.

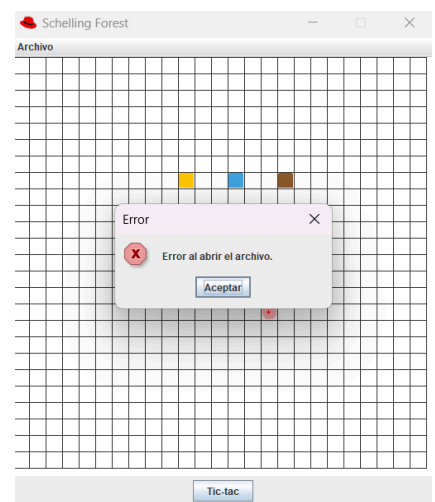
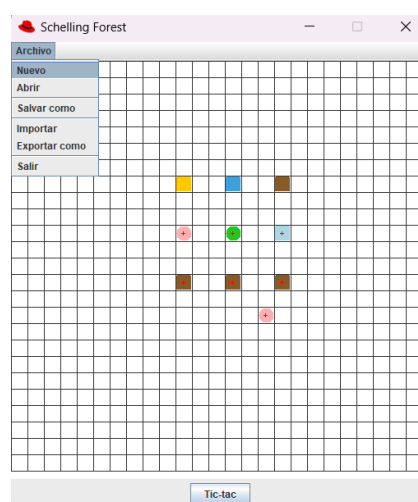
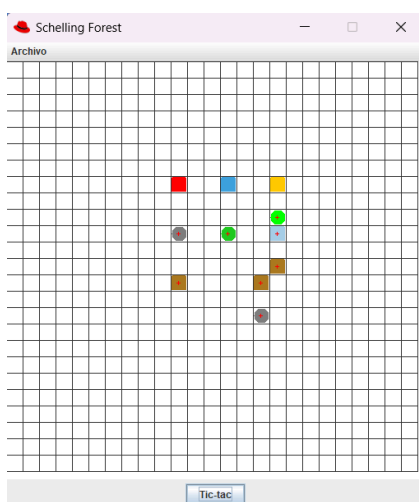
```
/**
 * Abre un bosque desde un archivo usando serialización.
 * Retorna la nueva instancia cargada para mantener las referencias internas correctas.
 */
public Forest open(File file) throws ForestException {
    try (java.io.ObjectInputStream in = new java.io.ObjectInputStream(new java.io.FileInputStream(file))) {
        return (Forest) in.readObject();
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_ABRIR);
    }
}
```

```
public class ForestException extends Exception {

    public static final String OPCION_OPEN_EN_CONSTRUCCION = "Opción open en construcción. Archivo ";
    public static final String OPCION_SAVE_AS_EN_CONSTRUCCION = "Opción saveAs en construcción. Archivo ";
    public static final String OPCION_IMPORT_EN_CONSTRUCCION = "Opción import en construcción. Archivo ";
    public static final String OPCION_EXPORT_AS_EN_CONSTRUCCION = "Opción exportAs en construcción. Archivo ";
    public static final String ERROR_GUARDAR = "Error al guardar el archivo.";
    public static final String ERROR_ABRIR = "Error al abrir el archivo.";

    public ForestException(String message) {
        super(message);
    }
}
```

5. Realicen una prueba de aceptación para este método iniciando la aplicación, creando un nuevo estado y abriendo el archivo `forestOne.dat`. Capturen imágenes significativas de estos resultados.



B Exportar e Importar

Estas operaciones nos van a permitir importar información de `forest` desde un archivo de texto y exportarlo. Los nombres de los archivos de texto deben tener como extensión `.txt`. Los archivos texto tienen una línea de texto por cada ítem. En cada línea asociada un elemento se especifica el tipo y la posición.

Tree 10, 20

Squirrel 15, 15

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import00` y `exportAs00`.

```
/**
 * Importa la configuración de un bosque (versión inicial).
 */
public void importFile00(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_IMPORT_EN_CONSTRUCCION + file.getName());
}

/**
 * Importa la configuración de un bosque.
 */
public void importFile(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_IMPORT_EN_CONSTRUCCION + file.getName());
}

/**
 * Exporta el estado del bosque a un formato específico (versión inicial).
 */
public void exportAs00(File file) throws ForestException {
    throw new ForestException(ForestException.OPCION_EXPORT_AS_EN_CONSTRUCCION + file.getName());
}
```

2. Construyan el método `exportAs` que ofrece el servicio de exportar a un archivo texto el estado actual, con el formato definido. Por ahora para las excepciones sólo consideren un mensaje de error general. No olviden diseño y pruebas de unidad.

```
/**
 * Exporta el estado del bosque a un formato específico (texto plano).
 */
public void exportAs(File file) throws ForestException {
    try (java.io.PrintWriter out = new java.io.PrintWriter(file)) {
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                Thing t = places[r][c];
                if (t != null) {
                    String tipo = t.getClass().getSimpleName();
                    out.println(tipo + " " + r + ", " + c);
                }
            }
        }
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_EXPORTAR);
    }
}
```

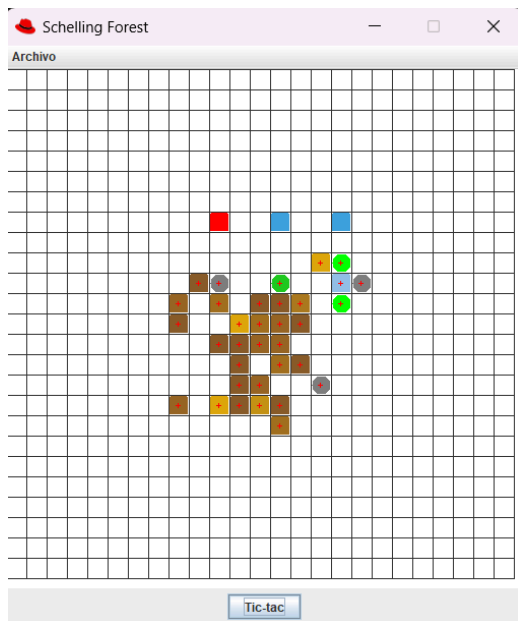
```
@Test
public void testExportAs() {
    try {
        Forest f = new Forest(true);
        new Fire(f, 10, 10);
        new Squirrel(f, 15, 15);
        java.io.File tempFile = java.io.File.createTempFile("testExport", ".txt");
        f.exportAs(tempFile);

        java.util.Scanner scanner = new java.util.Scanner(tempFile);
        String contenido = "";
        while(scanner.hasNextLine()) {
            contenido += scanner.nextLine() + "\n";
        }
        scanner.close();

        assertTrue("Debe contener Fire 10, 10", contenido.contains("Fire 10, 10"));
        assertTrue("Debe contener Squirrel 15, 15", contenido.contains("Squirrel 15, 15"));

        tempFile.delete();
    } catch (Exception e) {
        fail("Falló testExportAs: " + e.getMessage());
    }
}
```

3. Realicen una prueba de aceptación de este método: iniciando la aplicación y exportando como `forestOne.txt`. Editen el archivo y analicen los resultados. ¿Qué pasó?



```
Fire 7, 10
Water 7, 13
Water 7, 16
Squirrel 9, 15
Tree 9, 16
Squirrel 10, 9
Tree 10, 10
FireTree 10, 13
RainTree 10, 16
Tree 10, 17
Squirrel 11, 8
Squirrel 11, 10
Squirrel 11, 12
Squirrel 11, 13
Squirrel 11, 14
Tree 11, 16
Squirrel 12, 8
Squirrel 12, 11
Squirrel 12, 12
Squirrel 12, 13
Squirrel 12, 14
Squirrel 13, 10
Squirrel 13, 11
Squirrel 13, 12
Squirrel 13, 13
Squirrel 14, 11
Squirrel 14, 13
Squirrel 14, 14
Squirrel 15, 11
Squirrel 15, 12
Tree 15, 15
Squirrel 16, 8
Squirrel 16, 10
Squirrel 16, 11
Squirrel 16, 12
Squirrel 16, 13
Squirrel 17, 13
```

Al exportar el estado actual como `forestOne.txt` y abrirlo en un editor de texto, se genera un archivo de texto plano estructurado. Cada línea del archivo documenta un elemento específico de la matriz, especificando el tipo de objeto seguido por sus coordenadas de fila y columna correspondientes (por ejemplo: `Squirrel 13, 10`). A diferencia de la persistencia con `Serializable`, este formato permite visualizar directamente el estado de los objetos y modificar sus posiciones alterando los valores numéricos directamente en el archivo, lo que facilita el intercambio de configuraciones iniciales.

4. Construyan el método `import` que ofrece el servicio de importar de un archivo texto con el formato definido. Por ahora sólo considere un mensaje de error general. No olviden diseño y pruebas de unidad. (Consulten en la clase `String` los métodos `trim` y `split`)

```
/**
 * Importa la configuración de un bosque leyendo un archivo de texto.
 * Usa split para separar el tipo y la posición.
 */
public Forest importFile(File file) throws ForestException {
    try (java.io.BufferedReader in = new java.io.BufferedReader(new java.io.FileReader(file))) {
        Forest loadedForest = new Forest(true);
        String line;
        while ((line = in.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] parts = line.split("[,\\s]+");
            if (parts.length >= 3) {
                String tipo = parts[0];
                int r = Integer.parseInt(parts[1]);
                int c = Integer.parseInt(parts[2]);

                switch(tipo) {
                    case "Tree": new Tree(loadedForest, r, c); break;
                    case "FireTree": new FireTree(loadedForest, r, c); break;
                    case "RainTree": new RainTree(loadedForest, r, c); break;
                    case "Squirrel": new Squirrel(loadedForest, r, c); break;
                    case "Fire": new Fire(loadedForest, r, c); break;
                    case "Water": new Water(loadedForest, r, c); break;
                    case "Ground": new Ground(loadedForest, r, c); break;
                }
            }
        }
        return loadedForest;
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_IMPORTAR);
    }
}
```

```
/**
 * Permite seleccionar un archivo para importar una configuración.
 */
private void optionImport() {
    JFileChooser fileChooser = new JFileChooser();
    if (fileChooser.showOpenDialog(this) == JFileChooser.APPROVE_OPTION) {
        try {
            theForest = theForest.importFile(fileChooser.getSelectedFile());
            photo.repaint();
        } catch (ForestException e) {
            JOptionPane.showMessageDialog(this, e.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
        } catch (Exception e) {
            JOptionPane.showMessageDialog(this, "Ocurrió un error inesperado.", "Fatal Error",
                JOptionPane.ERROR_MESSAGE);
        }
    }
}
```

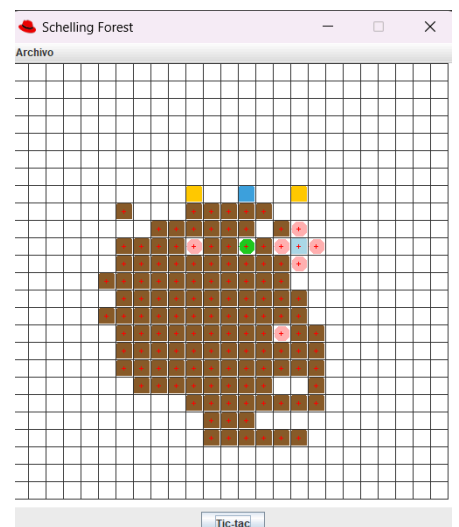
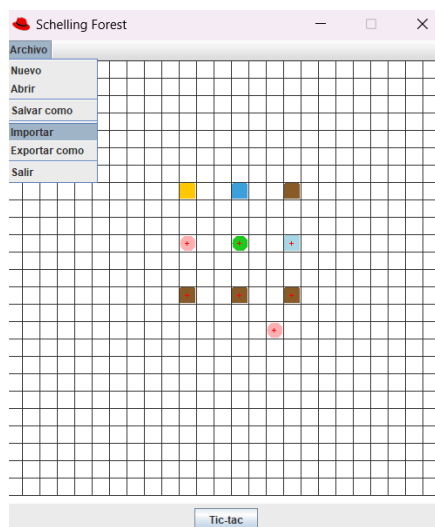
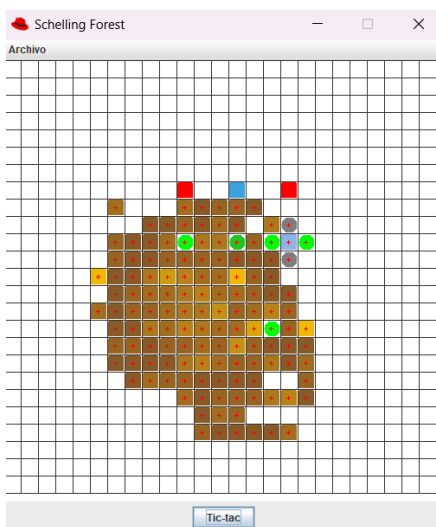
```
@Test
public void testImportFile() {
    try {
        java.io.File tempFile = java.io.File.createTempFile("testImport", ".txt");
        java.io.PrintWriter out = new java.io.PrintWriter(tempFile);
        out.println("Water 5, 5");
        out.println("Tree 12, 12");
        out.close();

        Forest f = new Forest(true);
        f = f.importFile(tempFile);

        assertNotNull("Debe haber Water", f.getThing(5, 5));
        assertTrue("Debe ser instancia de Water", f.getThing(5, 5) instanceof Water);
        assertNotNull("Debe haber Tree", f.getThing(12, 12));
        assertTrue("Debe ser instancia de Tree", f.getThing(12, 12) instanceof Tree);

        tempFile.delete();
    } catch (Exception e) {
        fail("Falló testImportFile: " + e.getMessage());
    }
}
```

5. Realicen una prueba de aceptación de este par de métodos: iniciando la aplicación exportando a `forest0ther.txt`, saliendo, entrando, creando una nueva e importando el archivo `forest0ther.txt`. ¿Qué resultado obtuvieron? Capturen la pantalla final.



Al realizar el ciclo completo de exportación e importación, el resultado obtenido fue la reconstrucción exacta del bosque. El sistema logró interpretar correctamente la información del archivo de texto plano y recreó dinámicamente cada objeto en su posición original. Esto demuestra que la persistencia a través de archivos de texto estructurados es totalmente funcional, permitiendo salvar y restaurar estados complejos sin depender exclusivamente de la serialización binaria de Java.

6. Realicen otra prueba de aceptación de este método escribiendo un archivo de texto correcto en `forestOne.txt` e importe este archivo. ¿Qué resultado obtuvieron? Capturen la pantalla.

```

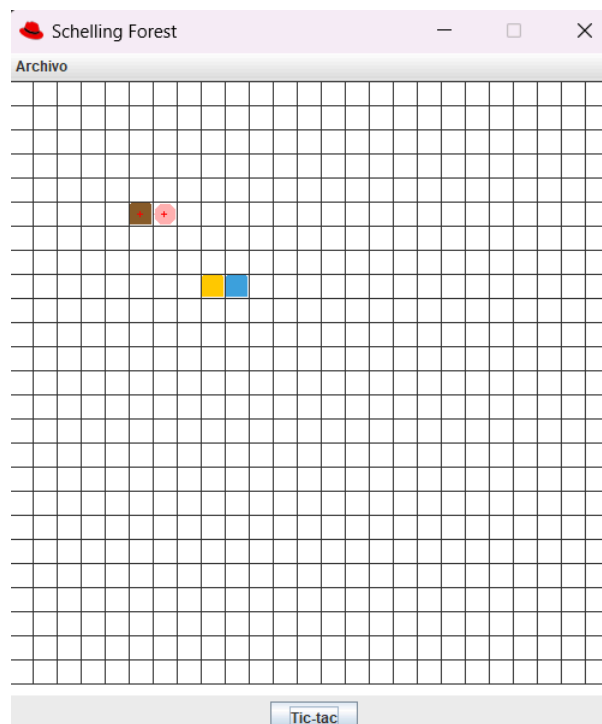
Archivo  Editar  Ver  H1  ≡  B  I  ...  ↗  ↘  👤
Fire 7, 10
Water 7, 13
Water 7, 16
Squirrel 9, 15
Tree 9, 16
Squirrel 10, 9
Tree 10, 10
FireTree 10, 13
RainTree 10, 16
Tree 10, 17
Squirrel 11, 8
Squirrel 11, 10
Squirrel 11, 12
Squirrel 11, 13
Squirrel 11, 14
Tree 11, 16
Squirrel 12, 8
Squirrel 12, 11
Squirrel 12, 12
Squirrel 12, 13
Squirrel 12, 14
Squirrel 13, 10
Squirrel 13, 11
Squirrel 13, 12
Squirrel 13, 13
Squirrel 14, 11
- - - - -
Ln 1, Col 1  553 caracteres.  Texto sin formatear  100%  Windows (CRLF)  UTF-8

```

```

Archivo  Editar  Ver  H1  ≡  B  I  ...  ↗  ↘  👤  ⚙️
Squirrel 5, 5
Tree 5, 6
Fire 8, 8
Water 8, 9
|

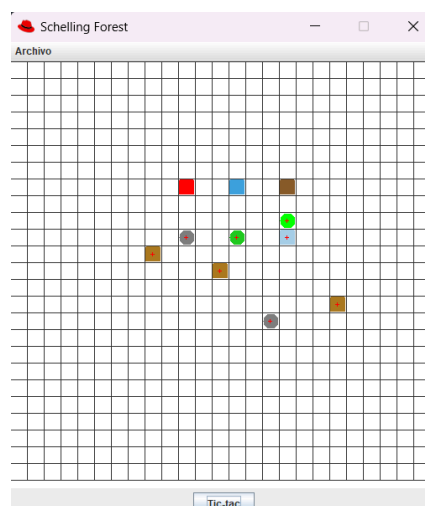
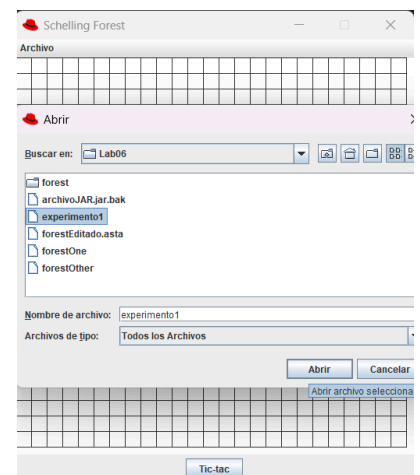
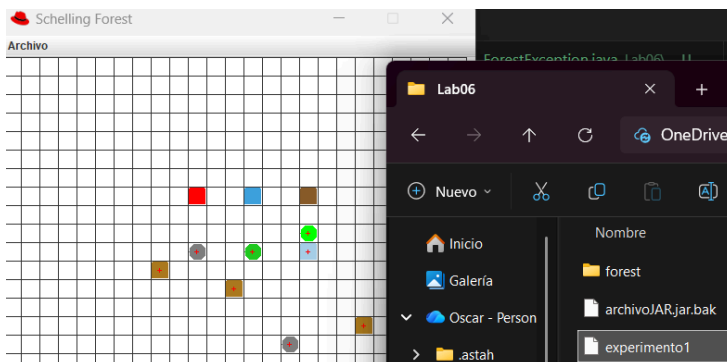
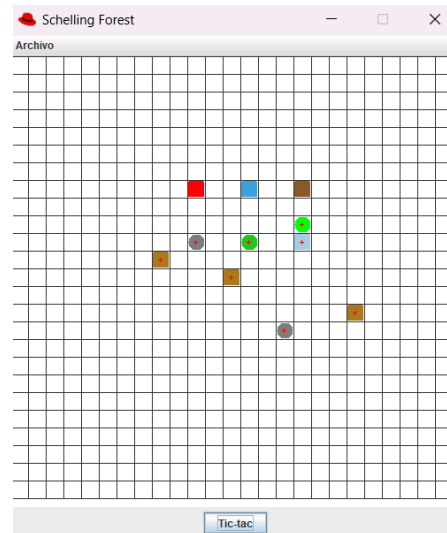
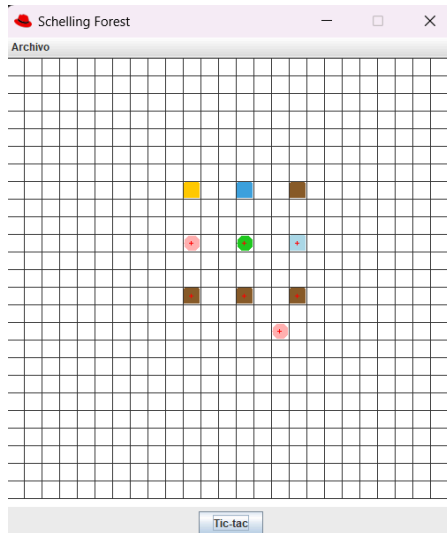
```



C Comportamiento

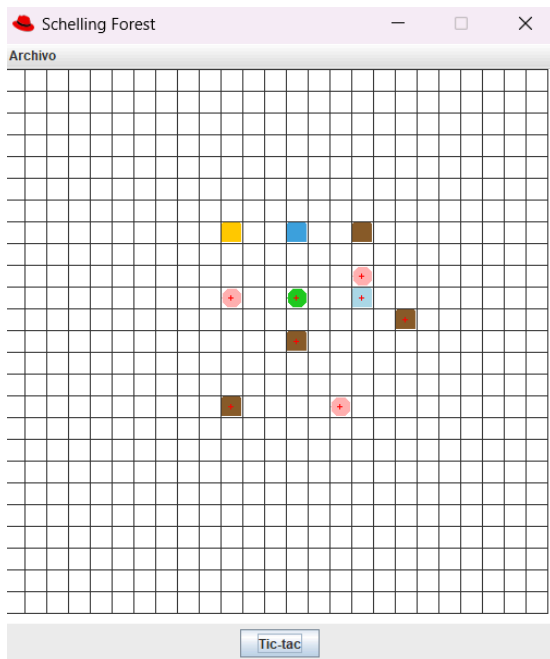
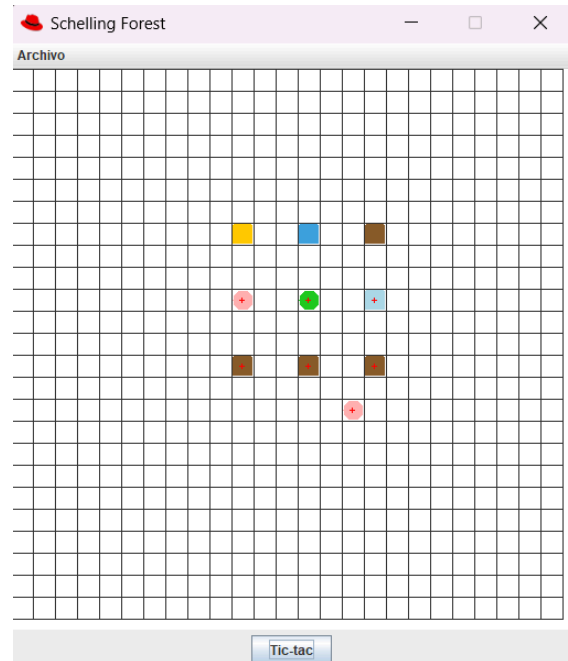
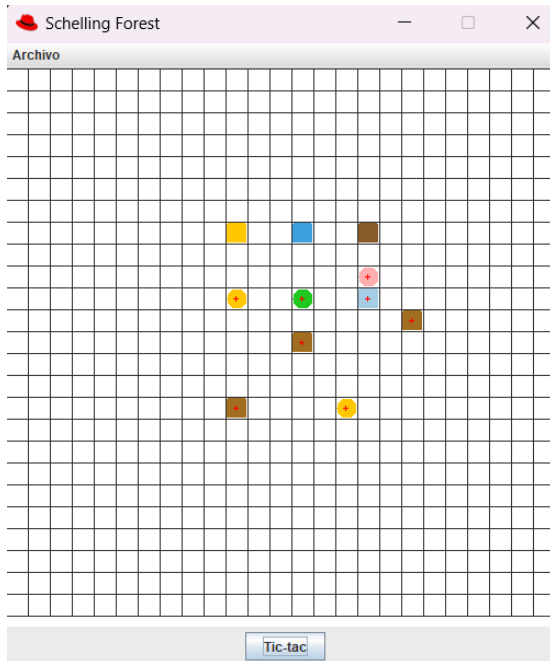
1. Ejecuten la aplicación, den tres clics, guarden a un archivo cualquiera y ábralo. Describan el comportamiento
2. Ejecuten la aplicación, tres clics, exporten a un archivo cualquiera e importen. Describan el comportamiento
3. ¿Qué diferencias ven el comportamiento 1. y 2.? Expliquen los resultados.

1.



El bosque continuó exactamente donde lo dejamos.

2.



- La principal diferencia es que el primer método guarda la memoria exacta de los objetos y el segundo no. Al usar el archivo .dat, el programa saca una copia fiel de la RAM, logrando que las ardillas conserven su edad, color y energía exactamente igual al reabrirlo. En cambio, con el archivo de texto solo guardamos las posiciones, lo que obliga al programa a crear los objetos de nuevo desde cero, haciendo que todos vuelvan a nacer con sus valores de fábrica (como si tuvieran 0 años).

PARTE III. Fortaleciendo la persistencia [En lab06.doc forest.astah *.java]

(NO OLVIDEN BDD – MDD)

A Guardar y abrir

- Copien las versiones actuales de `open` y `saveAs` y renómbrenlos como `open01` y `saveAs01`

```
/**
 * Versión 01 de open.
 */
public Forest open01(File file) throws ForestException {
    try (java.io.ObjectInputStream in = new java.io.ObjectInputStream(new java.io.FileInputStream(file)))
    {
        return (Forest) in.readObject();
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_ABRIR);
    }
}
```

```
/**
 * Versión 01 de saveAs.
 */
public void saveAs01(File file) throws ForestException {
    try (java.io.ObjectOutputStream out = new java.io.ObjectOutputStream(new java.io.FileOutputStream(file)))
    {
        out.writeObject(this);
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_GUARDAR);
    }
}
```

- Perfeccionen el manejo de excepciones de los métodos `open` y `saveAs` detallando los errores. No olviden pruebas de unidad.

```
public static final String ARCHIVO_NO_ENCONTRADO = "El archivo no existe.";
public static final String ARCHIVO_ES_DIRECTORIO = "La ruta indicada es un directorio.";
public static final String EXTENSION_INVALIDA_DAT = "El archivo debe terminar con la extensión .dat";
public static final String ERROR_FORMATO_OBJETO = "El archivo no contiene un bosque válido o está corrupto";
public static final String ERROR_CLASE_NO_ENCONTRADA = "No se pudo cargar la definición del bosque (clase)";

public ForestException(String message) {
    super(message);
}
```

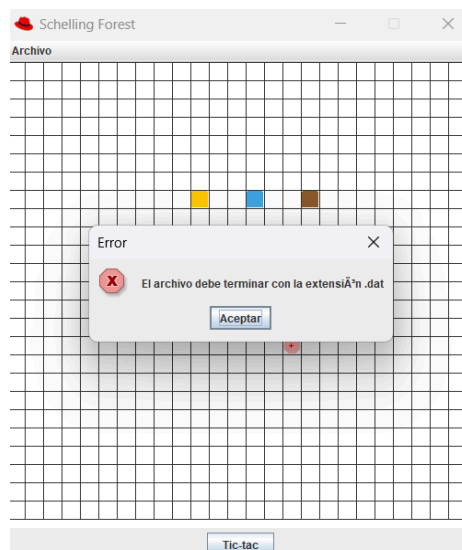
```
/**
 * Guarda el bosque actual en el archivo especificado usando serialización.
 */
public void saveAs(File file) throws ForestException {
    if (file.isDirectory()) {
        throw new ForestException(ForestException.ARCHIVO_ES_DIRECTORIO);
    }
    if (!file.getName().endsWith(".dat")) {
        throw new ForestException(ForestException.EXTENSION_INVALIDA_DAT);
    }

    try (java.io.ObjectOutputStream out = new java.io.ObjectOutputStream(new java.io.FileOutputStream(file))) {
        out.writeObject(this);
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_GUARDAR);
    }
}
}
```

```
@Test
public void testOpenFileNotFound() {
    try {
        Forest f = new Forest(true);
        f.open(new java.io.File("archivo_no_existente_test_xyz.dat"));
        fail("Se esperaba ForestException por archivo no encontrado.");
    } catch (ForestException e) {
        assertEquals(ForestException.ARCHIVO_NO_ENCONTRADO, e.getMessage());
    }
}

@Test
public void testSaveAsInvalidExtension() {
    try {
        Forest f = new Forest(true);
        f.saveAs(new java.io.File("bosque.txt")); // Not .dat
        fail("Se esperaba ForestException por extensión incorrecta.");
    } catch (ForestException e) {
        assertEquals(ForestException.EXTENSION_INVALIDA_DAT, e.getMessage());
    }
}
}
```

3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.



B Exportar e Importar

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import01` y `exportAs01`

```
/**
 * Versión 01 de importFile.
 */
public Forest importFile01(File file) throws ForestException {
    try (java.io.BufferedReader in = new java.io.BufferedReader(new java.io.FileReader(file))) {
        Forest loadedForest = new Forest(true);
        String line;
        while ((line = in.readLine()) != null) {
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] parts = line.split("[,\\s]+");
            if (parts.length >= 3) {
                String tipo = parts[0];
                int r = Integer.parseInt(parts[1]);
                int c = Integer.parseInt(parts[2]);

                switch(tipo) {
                    case "Tree": new Tree(loadedForest, r, c); break;
                    case "FireTree": new FireTree(loadedForest, r, c); break;
                    case "RainTree": new RainTree(loadedForest, r, c); break;
                    case "Squirrel": new Squirrel(loadedForest, r, c); break;
                    case "Fire": new Fire(loadedForest, r, c); break;
                    case "Water": new Water(loadedForest, r, c); break;
                    case "Ground": new Ground(loadedForest, r, c); break;
                }
            }
        }
        return loadedForest;
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_IMPORTAR);
    }
}
```

```
/**
 * Versión 01 de exportAs.
 */
public void exportAs01(File file) throws ForestException {
    try (java.io.PrintWriter out = new java.io.PrintWriter(file)) {
        for (int r = 0; r < SIZE; r++) {
            for (int c = 0; c < SIZE; c++) {
                Thing t = places[r][c];
                if (t != null) {
                    String tipo = t.getClass().getSimpleName();
                    out.println(tipo + " " + r + ", " + c);
                }
            }
        }
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_EXPORTAR);
    }
}
```

2. Perfeccionen el manejo de excepciones de los métodos `import` y `exportAs` detallando los errores.
No olviden pruebas de unidad.

```
public class ForestException extends Exception {

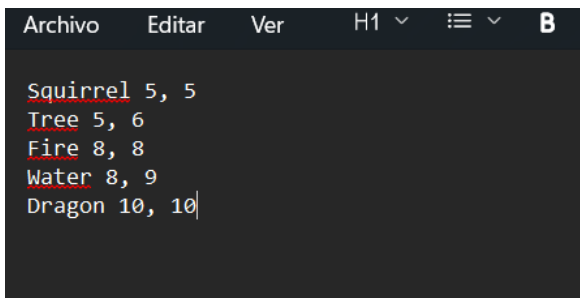
    public static final String OPCION_OPEN_EN_CONSTRUCCION = "Opción open en construcción. Archivo ";
    public static final String OPCION_SAVE_AS_EN_CONSTRUCCION = "Opción saveAs en construcción. Archivo ";
    public static final String OPCION_IMPORT_EN_CONSTRUCCION = "Opción import en construcción. Archivo ";
    public static final String OPCION_EXPORT_AS_EN_CONSTRUCCION = "Opción exportAs en construcción. Archivo ";
    public static final String ERROR_GUARDAR = "Error al guardar el archivo.";
    public static final String ERROR_ABRIR = "Error al abrir el archivo.";
    public static final String ERROR_EXPORTAR = "Error al exportar el archivo.";
    public static final String ERROR_IMPORTAR = "Error al importar el archivo.";

    public static final String ARCHIVO_NO_ENCONTRADO = "El archivo no existe.";
    public static final String ARCHIVO_ES_DIRECTORIO = "La ruta indicada es un directorio.";
    public static final String EXTENSION_INVALIDA_DAT = "El archivo debe terminar con la extensión .dat";
    public static final String ERROR_FORMATO_OBJETO = "El archivo no contiene un bosque válido o está corrupto";
    public static final String ERROR_CLASE_NO_ENCONTRADA = "No se pudo cargar la definición del bosque (clase ";

    public static final String EXTENSION_INVALIDA_TXT = "El archivo debe terminar con la extensión .txt";
    public static final String ERROR_LINEA_FORMATO = "Error en la línea: ";
    public static final String ERROR_TIPO_DESCONOCIDO = "Tipo de elemento desconocido: ";

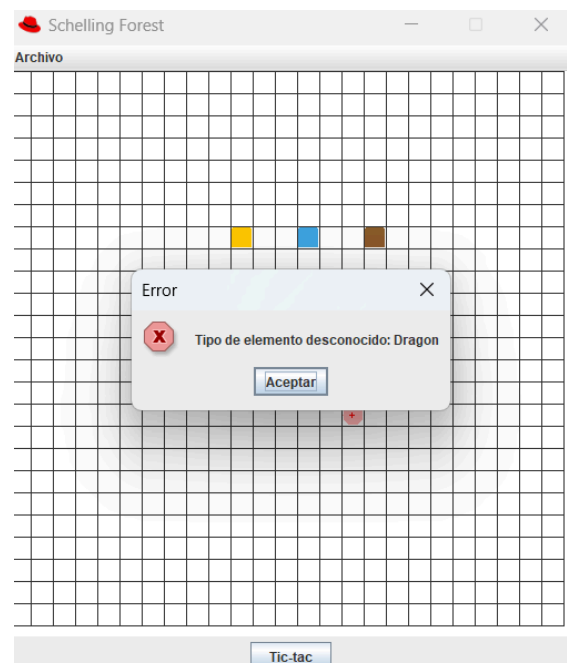
    public ForestException(String message) {
        super(message);
    }
}
```

3. Realicen una prueba de aceptación para validar uno de los nuevos mensajes diseñados, ejecútenla y capturen la pantalla final.



```
Archivo  Editar  Ver  H1  ≡  B

Squirrel 5, 5
Tree 5, 6
Fire 8, 8
Water 8, 9
Dragon 10, 10
```



PARTE IV. BONO. Perfeccionando importar [En lab06.doc forest.astah *.java]

(NO OLVIDEN BDD – MDD)

A Minicompilador

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import02` y `exportAs02`

```
/**
 * Importa la configuración de un bosque leyendo un archivo de texto.
 * Versión 02: Funciona como un minicompilador con errores detallados.
 */
public Forest import02(File file) throws ForestException {
    if (!file.exists()) throw new ForestException(ForestException.ARCHIVO_NO_ENCONTRADO);
    if (file.isDirectory()) throw new ForestException(ForestException.ARCHIVO_ES_DIRECTORIO);
    if (!file.getName().endsWith(".txt")) throw new ForestException(ForestException.EXTENSION_INVALIDA_TXT);

    try (java.io.BufferedReader in = new java.io.BufferedReader(new java.io.FileReader(file))) {
        Forest loadedForest = new Forest(true);
        String line;
        int numLinea = 0;
        while ((line = in.readLine()) != null) {
            numLinea++;
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] parts = line.split("[,\\s]+");
            if (parts.length < 3) {
                throw new ForestParseException(numLinea, line, "Faltan datos en la instrucción. Se esperaba: Tipo fila columna");
            }

            String tipo = parts[0];
            int r, c;
            try {
                r = Integer.parseInt(parts[1]);
            } catch (NumberFormatException e) {
                throw new ForestParseException(numLinea, parts[1], "La coordenada de la fila debe ser un número entero válido.");
            }
            try {

```

```
/**
 * Exporta el estado del bosque a un archivo de texto.
 * Versión 02
 */
public void exportAs02(File file) throws ForestException {
    exportAs(file);
}
```

2. Perfeccionen el método `import` para que, además de los errores generales, en las excepciones indique el detalle de los errores encontrados en el archivo (como un compilador) : número de línea donde se encontró el error, palabra que tiene el error y causa de error.

```

if (file.isDirectory()) throw new ForestException(ForestException.ARCHIVO_ES_DIRECTORIO);
if (!file.getName().endsWith(".txt")) throw new ForestException(ForestException.EXTENSION_INVALIDA_TXT);

try (java.io.BufferedReader in = new java.io.BufferedReader(new java.io.FileReader(file))) {
    Forest loadedForest = new Forest(true);
    String line;
    int numLinea = 0;
    while ((line = in.readLine()) != null) {
        numLinea++;
        line = line.trim();
        if (line.isEmpty()) continue;

        String[] parts = line.split("[,\\s]+");
        if (parts.length < 3) {
            throw new ForestParseException(numLinea, line, "Faltan datos en la instrucción. Se esperaba: Tipo fila columna");
        }

        String tipo = parts[0];
        int r, c;
        try {
            r = Integer.parseInt(parts[1]);
        } catch (NumberFormatException e) {
            throw new ForestParseException(numLinea, parts[1], "La coordenada de la fila debe ser un número entero válido.");
        }
        try {
            c = Integer.parseInt(parts[2]);
        } catch (NumberFormatException e) {
            throw new ForestParseException(numLinea, parts[2], "La coordenada de la columna debe ser un número entero válido.");
        }
    }
}

```

3. Escriban otro archivo con mínimo tres errores diferentes, llámenlo `forestErr.txt`, para ir arreglándolo con ayuda de su "importador". Presenten las pantallas que contengan los errores.

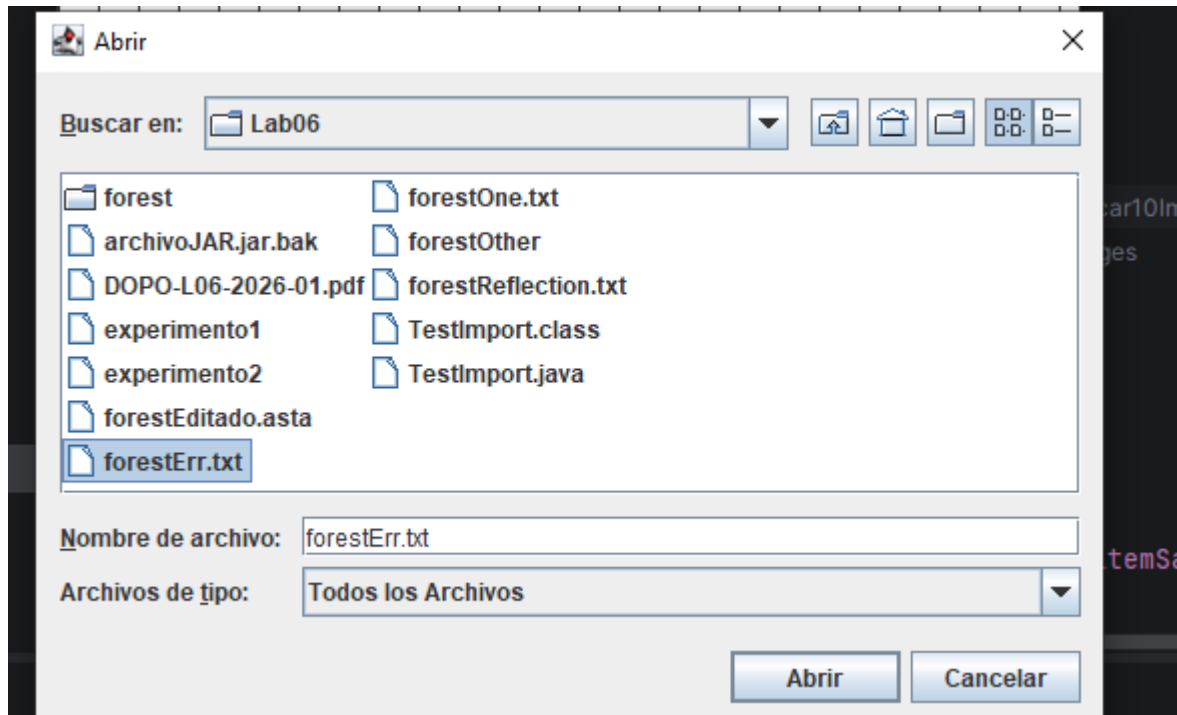
```

forestErr.txt
1  Tree 10 10
2  Dragon 12 15
3  Squirrel abc 10
4  FireTree 14
5  RainTree 15 15
6

```

Error en Dragon (No existe)

Error en Squirrel (Debe ser un número) y Error en FireTree (No puede ser nulo)



Error

X



Error de compilación en línea 2 | Palabra: 'Dragon' | Causa: Clase no encontrada en el dominio mediante Reflection. Tipo no soportado.

Aceptar

B Minicompilador flexible

1. Copien las versiones actuales de `import` y `exportAs` y renómbrenlos como `import03` y `exportAs03`
2. Perfeccionen los métodos `import` y `exportAs` para que pueda servir para cualquier tipo de elementos creados en el futuro. No olviden pruebas de unidad. (Investiguen cómo crear un objeto de una clase dado su nombre)
3. Escriban otro archivo de pruebas, llámelo `.txt`, para probar la flexibilidad. Presente una pantalla que contenga un error significativo.


```
public Forest import03(File file) throws ForestException {
    if (!file.exists()) throw new ForestException(ForestException.ARCHIVO_NO_ENCONTRADO);
    if (file.isDirectory()) throw new ForestException(ForestException.ARCHIVO_ES_DIRECTORIO);
    if (!file.getName().endsWith(".txt")) throw new ForestException(ForestException.EXTENSION_INVALIDA_TXT);

    try (java.io.BufferedReader in = new java.io.BufferedReader(new java.io.FileReader(file))) {
        Forest loadedForest = new Forest(true);
        String line;
        int numLinea = 0;
        while ((line = in.readLine()) != null) {
            numLinea++;
            line = line.trim();
            if (line.isEmpty()) continue;

            String[] parts = line.split("[,\\s]+");
            if (parts.length < 3) {
                throw new ForestParseException(numLinea, line, "Faltan datos en la instrucción. Se esperaba: Tipo fila columna");
            }

            String tipo = parts[0];
            int r, c;
            try {
                r = Integer.parseInt(parts[1]);
```

```
                c = Integer.parseInt(parts[2]);
            } catch (NumberFormatException e) {
                throw new ForestParseException(numLinea, parts[1] + " " + parts[2], "Las coordenadas deben ser números enteros.");
            }

            try {
                // API de Reflection: Instanciación dinámica
                Class<?> clazz = Class.forName("domain." + tipo);
                java.lang.reflect.Constructor<?> constructor = clazz.getConstructor(Forest.class, int.class, int.class);
                constructor.newInstance(loadedForest, r, c);
            } catch (ClassNotFoundException e) {
                throw new ForestParseException(numLinea, tipo, "Clase no encontrada en el dominio mediante Reflection. Tipo no soportado.");
            } catch (NoSuchMethodException e) {
                throw new ForestParseException(numLinea, tipo, "La clase no tiene un constructor (Forest, int, int).");
            } catch (Exception e) {
                throw new ForestParseException(numLinea, tipo, "Error al instanciar el elemento vía Reflection: " + e.getMessage());
            }
        }
        return loadedForest;
    } catch (ForestException e) {
        throw e; // Lanzar las excepciones parseadas hacia arriba
    } catch (Exception e) {
        throw new ForestException(ForestException.ERROR_IMPORTAR);
    }
}
```

Activar Windows

```
/**
 * Exporta el estado del bosque a un archivo de texto.
 * Versión 03
 */
public void exportAs03(File file) throws ForestException {
    exportAs(file);
}
```

PRUEBA UNITARIAS

```
public class ForestTestBonus {

    @Test
    public void testImport02DetailedException() {
        Forest forest = new Forest(true);
        File errFile = new File("../forestErr.txt");

        try {
            forest.import02(errFile);
            fail("Debería haber lanzado una excepción detallada ForestParseException");
        } catch (ForestParseException e) {
            // Evaluamos que la primera línea con error en forestErr.txt ("Dragon 12 15") sea reportada.
            assertEquals("Debe identificar el error en la línea 2", 2, e.getLineNumber());
            assertEquals("Debe identificar 'Dragon' como el causante", "Dragon", e.getErroneousWord());
            assertTrue("El mensaje debe indicar tipo desconocido", e.getMessage().contains("Palabra reservada"));
        } catch (ForestException e) {
            fail("La excepción debería ser de tipo ForestParseException");
        }
    }

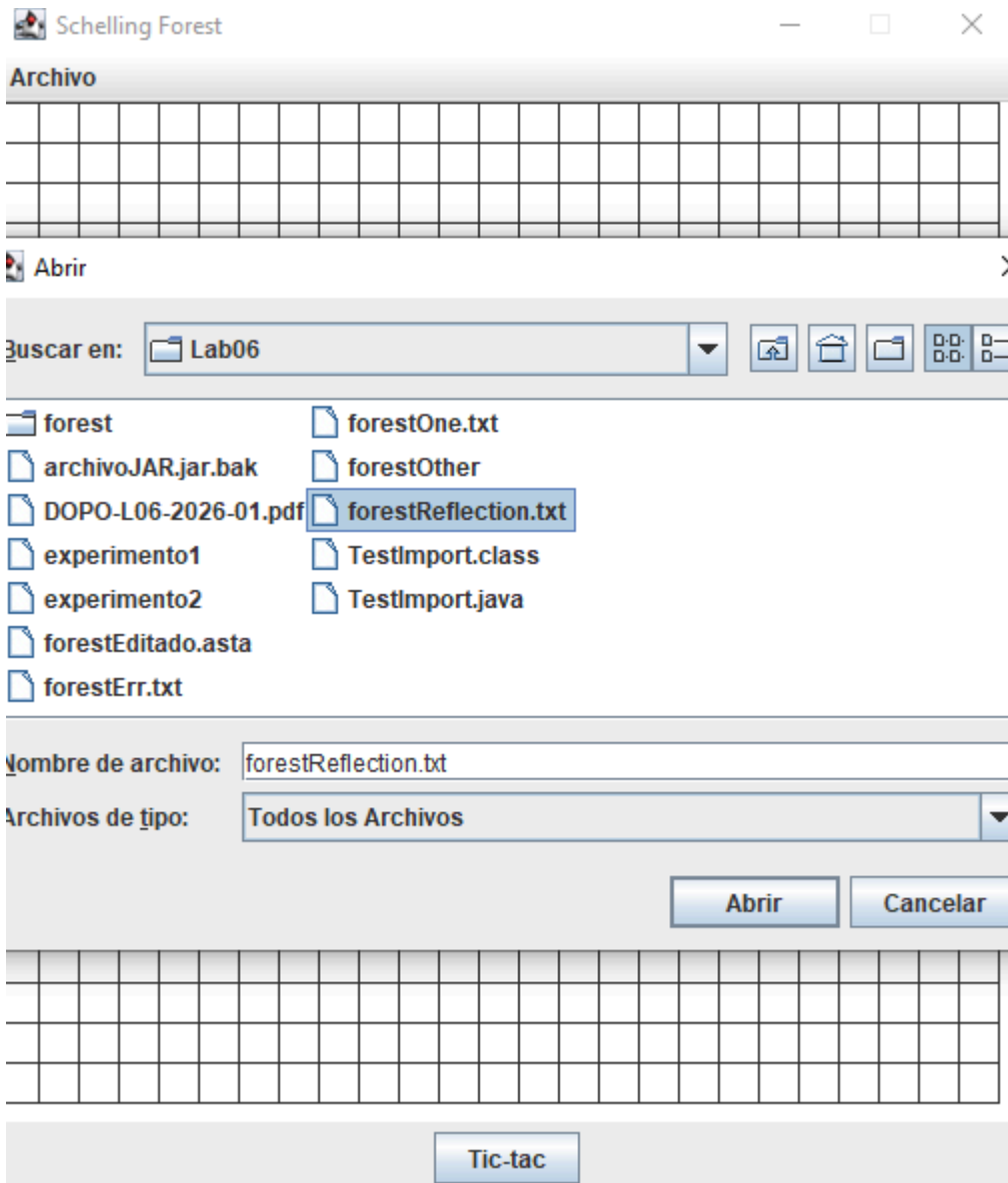
    @Test
    public void testImport03FlexibleReflection() {
        Forest forest = new Forest(true);
        File reflectionFile = new File("../forestReflection.txt");

        try {
            Forest loaded = forest.import03(reflectionFile);
            assertNotNull("El bosque debería cargarse exitosamente usando Reflection", loaded);

            // Verificamos que los objetos fueron instanciados (por ejemplo, Tree en 1,1)
            assertTrue(loaded.getThing(1, 1) instanceof Tree);
        }
    }
}
```

forestReflection.txt

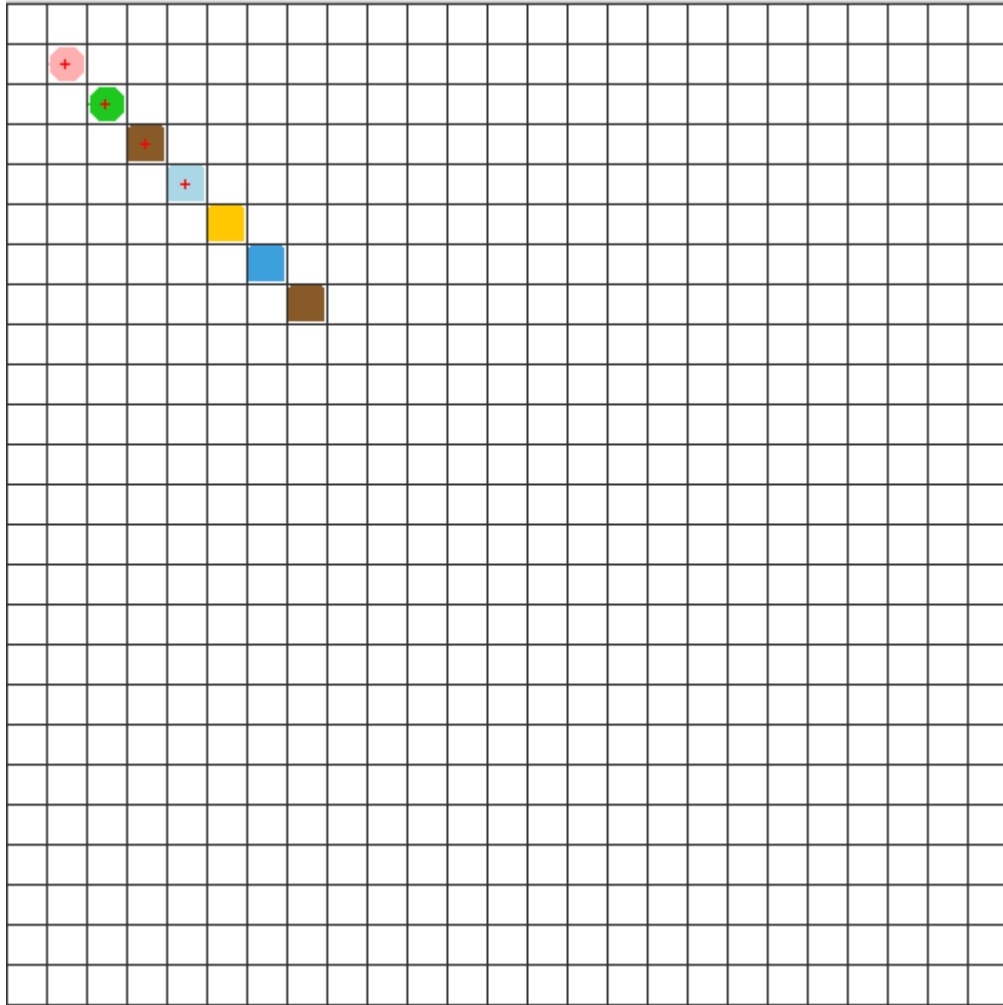
1	Tree	1	1
2	FireTree	2	2
3	Squirrel	3	3
4	RainTree	4	4
5	Fire	5	5
6	Water	6	6
7	Ground	7	7
8			



Schelling Forest



Archivo



Tic-tac